

Faculty of Computers and Artificial Intelligence

CS251 Introduction to Software Engineering

Dr : Mohammed El-Ramly

Masroofy

ID	Name	Email	Mobile
20240547	Mahmoud Ahmed Mohamed	20240547@stud.fci-cu.edu.eg	01050651009
20242152	Sara Mahmoud Mohamed Elbably	20242152@stud.fci-cu.edu.eg	01140643106
20242354	Mennatullah Mohamed Mahmoud	20242354@stud.fci-cu.edu.eg	01018083984
20242395	Yasmeen Nasridden Gad	20242395@stud.fci-cu.edu.eg	01154686378

Masroofy Overview

Masroofy is a smart expense and budget management application designed to help users organize their financial life in a simple and efficient way.

The system allows users to track daily expenses, manage budget cycles, monitor spending behavior, and calculate safe daily limits automatically.

Target Users

- Students
- Young adults
- Employees
- Anyone who wants better control over personal finances

Main Benefits

- Helps users manage their expenses effectively
- Tracks daily and monthly spending
- Prevents overspending through smart daily limits
- Organizes expenses into categories
- Provides a clear financial dashboard and reports

Project Goal

The main goal of Masroofy is to simplify personal financial management and encourage smarter spending habits using an organized and user-friendly system.

Budget & Expense Component – MVT Architecture

Model Layer

Responsible for data management and business logic.

Classes:

•BudgetCycle

- Manages the user's budget period.
- Tracks balance, spending limit, and remaining days.

•Transaction

- Stores expense transactions.
- Contains amount, category, and timestamp.

•Category

- Represents expense categories.

•DailyRecord

- Tracks daily spending and rollover calculations.

DAO Classes:

- CycleDAO
- TransactionDAO
- CategoryDAO
- DailyRecordDAO

These classes handle database operations such as insert, update, and retrieval.

View Layer

Handles user interaction and application flow.

Main Views:

BudgetSetupView

Creates a new budget cycle.

ExpenseView

Handles expense entry and validation.

DashboardView

Displays financial statistics and daily limits.

RolloverView

Processes daily rollover calculations.

Template Layer

Responsible for UI rendering and user interface elements.

Templates:

InitializationTemplate

ExpenseEntryTemplate

DashboardTemplate

These templates provide reusable UI components and user feedback handling.

Dashboard & Statistics Component – MVT Architecture

Model Layer

Handles data storage and statistical calculations.

Main Entities:

BudgetCycle

Stores budget information and remaining balance.

Transaction

Represents expense records and spending history.

Category

Defines expense categories with icons and colors.

DailyRecord

Tracks daily spending activity.

DAO Classes:

TransactionDAO

CategoryDAO

CycleDAO

DailyRecordDAO

DAO classes retrieve and process financial data from the database.

View Layer

Responsible for business logic and user interaction.

DashboardView

Loads dashboard information.

Calculates safe daily spending limits.

Detects budget warnings and final-day conditions.

StatsView

Processes expense statistics.

Aggregates transactions by category.

Calculates percentages.

Generates pie chart data.

Template Layer

Handles UI rendering and visual presentation.

DashboardTemplate

Displays:

Remaining balance

Daily limit

Warning indicators

Budget alerts

StatsTemplate

Displays:

Pie charts

Category legends

Total spending statistics

Empty-state messages

Notification & History Component – MVT Architecture

Model Layer

Responsible for storing notifications and transaction history data.

Main Entities:

NotificationLog

Stores alert records and notification status.

Prevents duplicate notifications.

BudgetCycle

Tracks spending percentage and budget status.

Transaction

Stores expense transactions.

Category

Represents expense classifications.

Enumeration:

AlertType

WARNING_80

EXHAUSTED_100

Defines alert levels based on budget consumption.

View Layer

NotificationView

Responsible for:

Monitoring budget thresholds.

Triggering warning notifications.

Prioritizing alerts.

Preventing duplicate alerts.

HistoryView

Responsible for:

Displaying transaction history.

Filtering by category and date.

Managing search and filtering operations.

Template Layer

NotificationTemplate

Displays:

Budget warning alerts.

Exhausted balance notifications.

HistoryTemplate

Displays:

Transaction lists.

Filter controls.

Empty-state messages

History Management Component – MVT Architecture

Model Layer

Responsible for transaction data management.

Transaction Entity

Stores:

- Expense amount
- Category name
- Timestamp
- Related budget cycle

DAO Classes

TransactionDAO

Handles:

- Updating transactions
- Deleting transactions
- Filtering transactions
- Retrieving transaction history

CycleDAO

Responsible for:

Recalculating daily spending limits after modifications.

View Layer

HistoryView

Controls:

- Loading transaction history
- Editing expenses
- Deleting transactions
- Applying filters
- PIN validation for secure operations
- Acts as the connection between UI and database logic.

Template Layer

HistoryTemplate

Responsible for displaying:

- Transaction lists
- Edit forms
- Delete confirmations
- Filter controls
- Empty-state messages

.

Some Screens from Website

Master Your Student Budget with Masroofy

Track expenses, set daily limits, and gain financial freedom without spreadsheets.

Get Started

Login

Smart Tools for Modern Students

Everything you need to manage your finances



Expense Tracking

Log expenses instantly



Daily Limit

Smart spending control



Dashboard

Visual insights



Alerts

Budget warnings

Masroofy

HomeLoginGet Started

Masroofy

BUDGET TRACKER

MAIN

New Expense

Dashboard

History

Stats

Notifications

Logout

Create Account

Sign Up

Already have an account? [Login](#)

Masroofy

HomeLoginGet Started

Sara Elbably

Login

Login

Don't have an account? [Sign up](#)

 Notifications



You're on track!
You have spent **21.7%** of your budget. Keep it up!

BUDGET USAGE

21.7%

MasroofyBUDGET TRACKER

MAIN

New Expense

Dashboard

History

INSIGHTS

Stats

Notifications

Logout

Sara Elbably

+ Log Expense

Dashboard

TOTAL BUDGET3000.0

SPENT650.0

REMAINING2350.0

DAILY LIMIT78.33

LATEST ACTIVITY

CATEGORY	AMOUNT	DATE
Shopping	500.0 EGP	May 07, 2026
Food	100.0 EGP	May 07, 2026
Transport	50.0 EGP	May 07, 2026

MasroofyBUDGET TRACKER

MAIN

New Expense

Dashboard

History

INSIGHTS

Stats

Notifications

Logout

Sara Elbably

Expense History

Daily limit reached — spent 650.00 of 100.00 EGP.
Daily Budget Alert

TOTAL TRANSACTIONS3

TOTAL SPENT650.00 EGP

CATEGORIES USED5

FILTER TRANSACTIONS

CategoryAll Categories

Datemm/dd/yyyy

SEARCH

RESET

TRANSACTIONS 3

#	AMOUNT	CATEGORY	DATE & TIME	ACTION
1	500.00 EGP	Shopping	May 07, 2026 — 18:32	
2	100.00 EGP	Food	May 07, 2026 — 18:31	
3	50.00 EGP	Transport	May 07, 2026 — 18:30	

MasroofyBUDGET TRACKER

MAIN

New Expense

Dashboard

History

INSIGHTS

Stats

Notifications

Logout

Sara Elbably

Spending Stats

By Category

Category	Amount
Shopping	80%
Food	10%
Transport	10%

Daily Spending

Date	Spending (EGP)
2026-05-07	650.00

MasroofyBUDGET TRACKER

MAIN

New Expense

Dashboard

History

INSIGHTS

Stats

Notifications

Logout

Sara Elbably

New Expense

REMAINING BALANCE2350.00 EGP

May 7, 2026 — June 5, 2026

EditReset

Daily limit reached — spent 650.00 of 100.00 EGP.
Daily limit

REMAINING2350.00 EGP

DAILY LIMIT100.00 EGP

SPENT TODAY650.00 EGP

Record Expense

Amount (EGP)0.00

Category

FoodHealthEducationTransport

Shopping

Today 5

Shopping500.00 EGP18:32

Food100.00 EGP18:31

Transport50.00 EGP18:30

Education70.00 EGP12:34

Health50.00 EGP12:33

Some Screens from Doc

API Documentation

```
class ExpenseView
    STATUS_OK
    STATUS_OVER_DAILY
    STATUS_OVER_CYCLE
    STATUS_ERROR
    tx_dao
    cat_dao
    record_expense()
```

budget.expense_view

expense_view.py — ExpenseView Business logic layer: validates and records a new expense, then updates DailyRecord and BudgetCycle remaining balance.

Flow:

1. Validate input (amount > 0, category exists, cycle active)
2. Insert Transaction via TransactionDAO
3. Update budget_dailyrecord.total_spent for today
4. Update budget_budgetcycle.remaining_balance
5. Return result dict (success / error / warning flags)

class ExpenseView:

Records a new expense and keeps the cycle + daily record in sync. All DB writes happen in a single atomic block.

```
STATUS_OK = 'ok'

STATUS_OVER_DAILY = 'over_daily_limit'

STATUS_OVER_CYCLE = 'over_cycle'

STATUS_ERROR = 'error'

tx_dao
```

API Documentation

```
class CycleManager
    get_active_cycle()

class TransactionManager
    get_by_cycle()
    get_total_by_cycle()
    sum_by_category()

class DailyRecordManager
    get_by_date()
```

budget.managers

class CycleManager(django.db.models.manager.Manager):

Provides helper queries for budget cycle management.

def get_active_cycle(self):

Retrieves the currently active budget cycle.

Finds the budget cycle that is marked as active (is_active=True). Since the application enforces only one active cycle per user, this returns the single active cycle or None if none exists.

Returns: The active BudgetCycle instance, or None if no active cycle exists.

Note: This method assumes business logic prevents multiple active cycles. If multiple exist due to data issues, it returns the first one found.

class TransactionManager(django.db.models.manager.Manager):

Provides transaction query and aggregation helpers.

def get_by_cycle(self, cycle_id):

Retrieves all transactions for a specific budget cycle.

Filters transactions by the provided cycle_id, returning all transactions that belong to that budgeting period.

Args: cycle_id: The primary key of the BudgetCycle to filter by.

API Documentation

CATEGORY_ICONS

```
class Category
    name()
    icon_res_id()
    icon_emoji
    get_spending_percentage()
    class Category.DoesNotExist
    class Category.MultipleObjectsReturned
    class Category.NotUpdated
    id()
    objects()
    transaction_set

class BudgetCycle
    user
    objects()
    total_allowance()
```

Contents

views.py — Masroofy (FIXED)

API Documentation

```
login_view()
signup_view()
logout_view()
home()
setup_cycle()
get_daily_alert()
get_cycle_alert()
delete_transaction()
record_expense()

class HistoryView
    get()
    post()

class DashboardView
    login_url
```

budget.models

CATEGORY_ICONS =

show

class Category(django.db.models.base.Model):

Represents a budget category and provides display icon metadata.

def name(unknown):

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

def icon_res_id(unknown):

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

icon_emoji

def get_spending_percentage(self, cat_total, grand_total):

def id(unknown):

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

budget.views

views.py — Masroofy (FIXED)

Fixes in this version:

1. Edit cycle → updates the current cycle without deleting transactions
2. Reset cycle → deletes all transactions and resets cycle dates
3. timedelta → uses days - 1 to correct 31-day calculations
4. History → applies accurate date filtering within the cycle range

def login_view(request):

Authenticate a user and establish an active session.

This view supports both GET and POST methods. On GET it renders the login page. On POST it validates the submitted credentials using Django's authentication system and, if valid, logs the user in and redirects to the dashboard.

Side effects: - Creates a session for authenticated users. - Emits Django authentication events used by middleware and signals.

Args: request (HttpRequest): The incoming request. Expected POST fields: - username (str): The username of the user. - password (str): The raw password.

Returns: HttpResponse: Rendered login page for GET or failed POST. HttpResponseRedirect: Redirect to dashboard on successful login, or back to login on failure.



Thank You